SERV RISC-V + Stochastic-Computing NN Accelerator — Interview Prep

Qualcomm — CPU Design Verification Engineer, Senior (RISC-V), Cork

This is a grounded, file-by-file analysis of your project, followed by interview-ready answers. I read the actual RTL and your thesis report/slides rather than guessing — where I flag a gap or a weakness, it's because I found it in the code, not because I'm assuming the worst.

PART 1 — Baseline Architecture: How SERV Actually Works

Before talking about what you *added*, you need to be crisp about what SERV *is*, because that's the foundation every question will be built on.

SERV is a bit-serial, in-order, non-pipelined RV32I core. There is no fetch/decode/ execute pipeline in the superscalar sense — there is one datapath, `W` bits wide (default `W=1`), and a 32-bit instruction/word is processed **serially, one bit (or W bits) per clock**, driven by a counter-based FSM (`serv_state.v`).

- **`serv_top.v`** — top-level integration: wires together decode, state, immediate decoder, two buffer registers, ALU, register-file interface, memory interface, CSR, optional debug/RVFI, optional compressed-instruction decoder, optional bus aligner.
- **`serv_state.v`** — the heart of the machine. A 5-bit counter (`o_cnt`) plus a small shift register (`cnt_r`) generate every timing strobe in the core: `cnt0`, `cnt1`, `cnt2`, `cnt3`, `cnt7`, `cnt11`, `cnt12`, `cnt_done`, `cnt0to3`, `cnt12to31`. Instead of pipeline stages, SERV uses **cycle position within a 32-cycle instruction "frame"** as its control mechanism — e.g. `cnt0` marks bit 0 of the word (LSB, where compare results resolve), `cnt_done` marks bit 31 (MSB, where the instruction retires). This is conceptually the bit-serial equivalent of a state machine walking through IF→ID→EX→WB, except all of that happens *within one counted pass*, and there is no overlap between instructions (no pipelining, hence **no data/structural/control hazards in the classic superscalar sense** — everything is naturally serialized by construction).
- **`serv_decode.v`** — combinational decode of the RV32I opcode into control signals (ALU op select, branch type, mem width/signedness, CSR op, etc.), pre-registered (`PRE_REGISTER`) to ease timing closure.
- **`serv_immdec.v`** — extracts `rs1`/`rs2`/`rd` addresses and decodes/streams out the immediate bit-serially as the instruction is walked.
- **`serv_bufreg.v`** / **`serv_bufreg2.v`** — the two general-purpose shift/accumulate registers that hold operand A (address/shift amount path) and operand B (second

ALU operand / load-store data), replacing what would be pipeline latches in a parallel core.

- `serv_alu.v` — the **original, conventional** bit-serial ALU (1 bit of add/sub/compare/bool per cycle, with the carry threaded through a 1-bit register `add_cy_r` across cycles — this *is* the ripple-carry adder, just spread over time instead of space).
- `serv_ctrl.v` — PC management: next-PC mux for sequential/branch/jump/trap/mret.
- `serv_csr.v` — machine-mode CSRs, exception/interrupt entry (`mcause`, `mepc`, `mstatus`, `mie`), the whole trap path.
- `serv_rf_if.v` / `serv_rf_ram.v` / `serv_rf_ram_if.v` — register file interface; since SERV is bit-serial, the "register file" is read/written serially too, and this layer arbitrates who owns read/write ports (ALU result, memory result, CSR result).
- `serv_mem_if.v` / `serv_aligner.v` / `serv_compdec.v` — load/store byte-lane handling, instruction-bus alignment, and (optional) RVC (compressed instruction) decompression.
- `serv_debug.v` — optional RVFI (RISC-V Formal Interface) hookup, used for formal/lockstep verification against a golden ISA model (this is the piece most directly relevant to "verification methodology" questions — even though *you* didn't modify it, you should be able to explain what it's for).

What SERV deliberately does not have, and you should say this plainly and confidently rather than dance around it: no pipeline, no branch predictor (branches are simply resolved late, near the end of the 32-cycle frame, and the next fetch is issued after resolution — cheap misprediction penalty is avoided by *not predicting*), no out-of-order execution, no register renaming, no caches, no MMU, no SIMD. It is the "world's smallest RISC-V core" by design — a few hundred LUTs, meant to be small and correct rather than fast. This matters a lot for how you frame this project in the interview (see Part 6).

PART 2 — What You Built: The Stochastic-Computing NN Extension

Your thesis added a **second, coexisting ALU and a small NN-inference coprocessor** that share the core's execution resource under FSM arbitration. This is genuinely a microarchitectural contribution, not a bolt-on peripheral, because it touches the ALU input/output bus itself.

2.1 Dual-ALU arbitration (the core modification, in `serv_top.v`)

```
cpu_alu_en        = cnt_en & ~nn_alu_en          // CPU yields the ALU when
NN wants it
final_alu_ctrl_bus = nn_alu_en ? nn_alu_ctrl_bus : cpu_alu_ctrl_bus
final_alu_data_bus = nn_alu_en ? nn_alu_data_bus : cpu_alu_data_bus
```

```
final_alu_en        = nn_alu_en | cpu_alu_en
alu_rd              = USE_STOC_ALU ? alu_rd_stoc : alu_rd_conv
```

This is a **priority multiplexer / resource-arbitration scheme**, structurally identical in spirit to how a shared execution port is arbitrated between issue queues in a larger core — just with two requesters (CPU datapath, NN FSM) instead of N. `serv_alu` (original) and `serv_stoc_alu` (yours) are instantiated **in parallel**, both are fed the arbitrated bus, and a static `USE_STOC_ALU` parameter selects which one's *deterministic* result feeds the register file (the stochastic ALU has entirely separate stochastic opcodes it alone services, selected by `i_matrix_op`/`i_rd_sel`).

2.2 `serv_stoc_alu.v` — the ALU you designed

- Takes a 5-bit `i_alu_ctrl_bus` (`{rd_sel[2:0], bool_op[1:0]}`), a 3-bit compare-control bus, and a packed 4×W data bus (`{rs1, op_b, buf, mat_element}`).
- Implements stochastic arithmetic using the classic SC primitives: **AND gate = multiplication, OR gate = addition (unipolar, saturating), XOR = subtraction**, plus a large opcode space of NN-relevant approximations (sigmoid/tanh/ReLU/leaky-ReLU/swish/GELU/softmax, dot-product/convolution/pooling helpers, matrix add/mult/transpose, dropout, stochastic rounding, Gaussian-noise approximation) built out of 1-bit combinational logic plus a free-running on-die 8-bit LFSR (`lfsr <= {lfsr[6:0], lfsr[7]^lfsr[5]^lfsr[4]^lfsr[3]}`, seeded `8'hA5`) that supplies randomness for the probabilistic functions.
- Output is registered (`always @(posedge clk or posedge rst)`), one opcode resolved per cycle, selected via a `case({i_matrix_op, i_rd_sel})` with a secondary "auxiliary" bank selected when the top 2 control bits are `11` — i.e. you doubled the effective opcode space by overloading the same 5-bit control field with a mode bit, the same trick `serv_alu` uses with `i_bool_op` (`01` reused as "pass zero" during shifts).

2.3 Stochastic bitstream infrastructure

- `lfsr_rng.v` — parametrizable Fibonacci LFSR with maximal-length taps hardcoded for 8/16/32-bit widths, and a **non-maximal fallback** (`default:` case, just 2 taps) for any other width.
- `binary_to_stochastic_converter.v` — the standard SC encoder: on each cycle, compares the binary value against a fresh LFSR random sample; if `binary_in >= random_val`, emit a `1` into the bitstream. This is the textbook comparator-based unipolar Bernoulli encoder. It's inherently **serial and O(STOCHASTIC_BITS) cycles** — a direct architectural echo of SERV's own bit-serial philosophy, which is a nice narrative point: SC and SERV are a natural fit because both trade time (cycles) for area.
- `stochastic_to_binary_converter.v` — the decoder: a running popcount over the bitstream (`one_count`), scaled back to binary at the end (`one_count * (2^W-1) /`

`STOCHASTIC_BITS`).

- **`stochastic_relu.v`** / **`relu_binary.v`** — SC-domain and binary-domain ReLU implementations (your NN FSM actually uses both a stochastic activation path and a conventional-ALU compare-and-mux ReLU path — see below).
- **`select_max.v`** — sequential argmax over the final layer's outputs (10-neuron MNIST classification: iterates all neurons, tracks running max + index, asserts `layer_done`).
- **`avg_pooling.v`** — average-pooling reduction stage for the CNN-style pipeline.

2.4 NN inference FSM — `serv_nn_layer.v` / `serv_multi_layer_nn_top.v` / `serv_nn_input_layer.v` / `serv_nn_hidden_layer1.v` / `serv_nn_output_layer.v` / `serv_multi_layer_nn_top.v`

`serv_nn_layer` is a **Moore-ish FSM that drives the shared ALU bus directly**, exactly like a microcoded sequencer. Per output neuron it walks:

`IDLE → MAC_STOC → ACCUM_STOC → (loop over inputs) → ADD_BIAS_STD → ACTV_SIGMOID_STOC → ACTV_RELU_STD → NEXT_NEURON → …`

Notably it **deliberately runs the MAC twice, in two domains, every time**: once through the stochastic ALU (AND-multiply, accumulated in `accum_stoc`) and once through the conventional path (`accum_std`, plain binary add), and *both* accumulators are carried forward — sigmoid activation is taken from the stochastic accumulator, ReLU is taken from the standard accumulator via a compare-and-select on the conventional ALU (`o_alu_cmp_ctrl_bus`/SLT). This dual-domain execution is exactly the kind of thing that should jump out in a design-verification interview: it's a deliberate redundant computation used to let you **cross-check stochastic results against a deterministic reference in hardware**, which is good verification instinct, but it also means the two accumulators can (and will) diverge because SC is inherently approximate — worth having a crisp answer ready for "why compute it twice?" (see Part 4).

Weights/biases are hardcoded constants set at reset (`weights_arr`, `bias_arr`) — this is a fixed, not-yet-trained/loadable network, good to be upfront about if asked.

`serv_multi_layer_nn_top.v` chains input/hidden/output layer instances and re-exposes the same `o_alu_*`/`i_alu_rd` handshake up to `serv_top`, so from the CPU's point of view the whole multi-layer network is just "one more requester" on the arbitrated ALU bus.

2.5 System integration / IP wrapping (PYNQ-Z2, Vivado)

- **`axi_stream_to_parallel.v`** / **`parallel_to_axi_stream.v`** — AXI-Stream ⇄ parallel bus shims so the core's serial data can be moved on/off chip via Zynq's PS↔PL AXI fabric.
- **`parallel_serial_if.v`** / **`parallel_serial_if_unmod.v`** — parallel/serial conversion at the register-file boundary (`_unmod` suggests you kept an untouched baseline copy

alongside your modified version — good practice for A/B regression comparison, worth mentioning explicitly as a verification habit).

- `serv_rf_axi_stream_wrapper.v` / `serv_rf_axi_wrapper.v` / `serv_axi_lite_wrapper.v` / `serv_rf_top_serial_wrapper_unmod.v` — top-level AXI-Lite/AXI-Stream wrappers used to drop the modified core into the Zynq PS/PL flow for on-hardware validation.
- `top_serv_system.v` — full system top for synthesis/bitstream generation.

2.6 Testbenches you wrote

`serv_stoc_alu_tb.v` / `serv_stochastic_alu_tb.v` (directed opcode sweep of the new ALU), `tb_serv_neural_network.v` and `tb_serv_nn_layer.v` (system-level NN inference tests with categorized test suites — basic functionality, boundary values (zero/max/min input), random inputs, single-bit/alternating/sequential patterns, rapid-fire and back-to-back issue, reset-during-operation, start-while-busy, timing/setup-hold, timeout), plus `tb_serv_decode.v`, `tb_serv_immdec.v`, `tb_serv_state.v`, `tb_serv_rf_ram.v`, `tb_serv_rf_ram_if.v`, `tb_serv_top.v` exercising the surrounding baseline modules.

Be ready to discuss this honestly, because it's the most interview-relevant finding in the whole codebase: in `tb_serv_neural_network.v`, the pass/fail counting infrastructure is fully built (`test_count`/`pass_count`/`fail_count`, a `check_results` task comparing DUT outputs against `expected_relu`/`expected_sigmoid`/`expected_tanh`), but the three reference functions that are supposed to *compute* those expected values (`calculate_expected_relu`, `calculate_expected_sigmoid`, `calculate_expected_tanh`) are **empty stubs** — the scoreboard shape exists, the golden model behind it doesn't. This is not a knock on you — recognizing it and being able to say so plainly, unprompted, is *exactly* the "honest about what I know" trait Qualcomm is testing for. Own it (see Part 4/5).

PART 3 — The Six Standard Questions, Answered

1. What was the objective? Extend SERV — the smallest RV32I core available — with a second, dual-mode ALU that adds native stochastic-computing arithmetic (bitwise AND/OR/XOR as multiply/add/subtract) and a lightweight NN-inference datapath, while preserving SERV's core value proposition: minimal area, RV32I compliance, and bit-serial simplicity. Target platform: PYNQ-Z2 FPGA. Target application: ultra-low-power edge inference (MNIST-scale MLP/CNN), evaluated on resource overhead, power, and accuracy vs. bitstream length.

2. What exactly did you contribute?

- Designed `serv_stoc_alu`, a new bit-serial ALU implementing SC primitives and ~20 NN/SC opcodes, integrated it alongside the untouched `serv_alu` via a static output mux.

- Designed the ALU-bus arbitration logic in `serv_top` that lets an external FSM (the NN accelerator) seize the shared ALU on a cycle-by-cycle basis without corrupting in-flight CPU state (`cpu_alu_en = cnt_en & ~nn_alu_en`).
- Designed the NN inference sequencer (`serv_nn_layer`, chained into `serv_multi_layer_nn_top`), which reuses the arbitrated ALU to perform MAC, bias-add and activation in both a stochastic and a conventional numeric domain.
- Built the SC bitstream codec chain (`lfsr_rng`, `binary_to_stochastic_converter`, `stochastic_to_binary_converter`) and supporting NN primitives (`select_max`, `avg_pooling`, `stochastic_relu`/`relu_binary`).
- Wrapped the system for AXI/Zynq integration and wrote the testbenches and directed test suites listed above; synthesized on PYNQ-Z2 in Vivado and validated in ModelSim.

3. What was the hardest challenge? Two, and pick whichever the interviewer's follow-ups seem to want:

- *Architectural*: making two structurally different ALUs (event-driven combinational bit-serial arithmetic vs. registered stochastic-opcode arithmetic) share one control/data bus and one arbitration point without the CPU's in-flight instruction state (the `add_cy_r` carry register, the `cnt0` compare register) getting corrupted when the NN FSM borrows the ALU mid-instruction. That's why `cpu_alu_en` is gated by `~nn_alu_en` rather than the two paths running independently.
- *Numerical*: stochastic computing accuracy is inherently a function of bitstream length — going from 256 to 1024 bits trades latency for <1% mean error — and because SERV is bit-serial by nature, that latency cost compounds directly with the core's already-serial 32-cycles-per-word baseline. Reconciling "make SERV smaller/simpler" with "SC needs long streams for accuracy" was the central design tension of the whole thesis (this is explicitly Chapter 8.1 of your report — know it cold).

4. How did you debug it? Directed opcode sweeps against `serv_stoc_alu` in isolation first (`serv_stoc_alu_tb`) to pin down each new opcode's truth table before it ever touched the arbitrated bus, then integration-level testbenches (`tb_serv_nn_layer`, `tb_serv_neural_network`) covering boundary values, bit-pattern edge cases, back-to-back/rapid-fire issue, and reset-during- operation — i.e. you were explicitly hunting for the FSM-arbitration corner cases (what happens if a reset lands mid-MAC, what happens if `nn_start` reasserts while busy) rather than only checking the happy path. Waveform inspection (ModelSim `$dumpvars`) plus resource/timing closure checks in Vivado for the hardware side. *(Be ready to admit: the functional self-checking on the NN testbench never got a real reference model wired in — see Part 5 — so debugging there was closer to visual waveform/`$display` inspection than true automated regression. Say this before they find it.)*

5. If you started again today, what would you improve?

- Replace the empty `calculate_expected_*` stubs with an actual golden reference model (even a simple fixed-point C/Python MLP forward-pass) so `check_results` is a real scoreboard instead of a shape without a brain.
- Add SystemVerilog assertions at the arbitration boundary — there currently isn't a single `assert` in this codebase; correctness relies entirely on directed simulation. Minimum-viable set: mutual exclusion on `cpu_alu_en`/`nn_alu_en`, no register-file write contention between CPU and NN write-backs, LFSR-never-all-zero, bitstream converter's `conversion_done` pulsing exactly once per `STOCHASTIC_BITS` cycles.
- Decorrelate the two random sources: `serv_stoc_alu`'s internal 8-bit LFSR (seed `0xA5`) and the separate `lfsr_rng` instance driving `binary_to_stochastic_converter` are independent generators with no shared seed management — in stochastic computing, correlation between the bitstreams feeding an AND/OR gate silently biases the result (this is *the* classic SC pitfall — see Part 4, it's a strong thing to raise proactively).
- Add coverage collection (functional coverage on opcode space actually exercised, cross-coverage on "NN active while CPU mid-instruction") — right now correctness is argued from a fixed list of directed tests, not from coverage closure.

6. What did you learn? That resource arbitration is where the *real* bugs live, not inside either ALU individually — both `serv_alu` and `serv_stoc_alu` are individually simple and easy to verify in isolation, but the priority mux and the `cnt_en`-gating logic in `serv_top` is where a subtle timing assumption (whose result reaches the register file this cycle) can silently break either the CPU or the accelerator. Also learned, empirically, the precision/latency/area trilemma that stochastic computing forces on a bit-serial core — which is exactly the kind of trade-off reasoning CPU microarchitecture work in general requires (pick two of speed/power/area, justify the third you gave up, quantify it).

PART 4 — "Gotcha" Follow-Ups, With Answers Ready

"Why did you implement the dual-ALU as two full parallel instances + a mux, instead of modifying `serv_alu` in place?" Because it isolates risk: the original `serv_alu` stays byte-for-byte the golden, previously-verified baseline. Any regression in ordinary RV32I execution can be attributed to the arbitration/mux logic, never to accidental damage inside the trusted ALU. It also means `USE_STOC_ALU=0` gives you an instant, single-parameter fallback to stock SERV for A/B comparison — which is exactly what you did (Table 5.1 in the report compares `serv_alu` vs `serv_stoc_alu` feature-by-feature).

"What was the hardest bug?" Point to the reset wiring on `serv_stoc_alu` — the code literally still carries the comment `// ADD THIS LINE to connect the main reset` next to `.rst(i_rst)` in the instantiation, meaning at some point the stochastic ALU's internal LFSR and output register were *not* tied to the system reset. An un-reset LFSR/output register in a shared-resource design is a nasty class of bug: it doesn't fail every run, it fails

depending on power-up/simulation X-state, which is a very believable "hardest bug" story and it's literally evidenced in your own source — much stronger than inventing one.

"Why did your scoreboard work like that?" Be direct: the test *structure* (categorized directed suites, pass/fail counters, boundary/stress/error/timing buckets) reflects deliberate coverage planning across functional categories. But the self-checking oracle for the NN testbench was left as a placeholder — the comparison logic (`check_results`) is real, the reference-model functions it compares against are not yet implemented. If pushed on why you'd ship that: time-boxing under thesis deadlines meant validating the RTL against waveform/ hand-calculated spot checks first, with the intent to backfill an automated golden model that didn't happen before submission. That's a normal, honest answer — don't over-apologize for it, just be accurate.

"Why was an assertion needed [in the arbitration logic]?" Because `cpu_alu_en = cnt_en & ~nn_alu_en` is a *soft* guarantee enforced by how the signal happens to be wired, not a checked invariant — nothing currently detects if a future change (e.g. someone adding a third ALU requester, or reworking `cnt_en` timing) accidentally lets `cpu_alu_en` and `nn_alu_en` overlap and both drive the shared ALU bus in the same cycle. An assertion (`assert property (@(posedge clk) !(cpu_alu_en && nn_alu_en))`) turns that implicit assumption into something that fails loudly in simulation the moment it's violated, instead of silently corrupting a register-file write. This is the standard justification pattern for any control-path mutual-exclusion assertion — know it, it generalizes far beyond this specific project (issue-port arbitration, write-back port arbitration, anything with a shared resource and two requesters).

"How did your ALU modification work?" `serv_stoc_alu` takes the same shape of control/data bus as the original `serv_alu` (packed control bits + a 4×W data bus of `{rs1, op_b, buf, mat_element}`), so it drops into the same arbitrated bus without needing new plumbing. Internally it's purely combinational SC primitives (AND=multiply, OR=add, XOR=subtract) plus a registered output mux selected by `{i_matrix_op, i_rd_sel}`, with an on-die LFSR supplying the pseudo- randomness the probabilistic opcodes (sigmoid/tanh/dropout/Gaussian-noise approximations) need. One opcode resolves per clock, same as the conventional ALU resolves one bit-slice per clock — so from the FSM's point of view both ALUs have identical latency characteristics, which is what made them pluggable behind the same mux/arbitration point in the first place.

PART 5 — Honest Gaps (know these before they ask)

1. **No golden reference model behind the NN scoreboard** (see above) — the infrastructure to auto-check is there, the model isn't.
2. **Zero SystemVerilog assertions anywhere in the project.** All correctness claims rest on directed simulation plus `$display`/waveform inspection. For a role centered on *verification*, be ready to name what you'd add (mutual exclusion on ALU arbitration,

RF write-port contention, LFSR non-degeneracy, converter timing invariants) rather than claim the project already has it.

3. **Potential correlation between the two independent LFSR instances** used for bitstream generation vs. in-ALU stochastic activation approximation — correlated random sources feeding SC AND/OR gates bias the statistical result; this wasn't measured or addressed in the current design. Raising this unprompted is a strong signal of genuine SC understanding, not just RTL-following.
4. **This is not exposed as a RISC-V ISA extension.** The NN accelerator is triggered via an external `nn_start` pin/AXI wrapper signal and directly seizes the ALU bus — it is *not* wired through `serv_decode`'s custom-opcode path or `EXT_ENABLED`/MDU hooks that already exist in `serv_top` for that purpose. Be precise about this if asked "is this a custom instruction?" — it isn't; it's a bus-level coprocessor that borrows a shared execution unit, architecturally closer to a tightly-coupled accelerator than an ISA extension. (A very reasonable "if I did this again" answer: expose it as a real custom R-type opcode through `serv_decode`, so it's issued and verified exactly like any other RV32I instruction, including through RVFI.)
5. **Weights/biases are hardcoded at reset**, not loaded — there's no trained-model load path.

Being the one to volunteer all five of these, calmly, is worth more in this interview than pretending the project is airtight.

PART 6 — Bridging This Project to a Superscalar/OoO CPU-DV Role

Be upfront about the honest mapping: SERV has **no pipeline, no branch predictor, no renaming, no OoO issue, no caches, no MMU, no SIMD** — so this project alone does not demonstrate hands-on verification of those structures. That's fine to say plainly. What it *does* demonstrate, and what you should actively steer the conversation toward:

- **Shared-resource arbitration** (your ALU mux) is the same *category* of problem as execution-port arbitration, write-back port contention, or LSQ/ROB structural hazards in an OoO core — smaller scale, same reasoning discipline (identify the shared resource, identify all requesters, prove mutual exclusion, decide what "correct" means when two requesters collide).
- **Bit-serial timing (`serv_state`'s counter FSM)** is a clean way to talk fluently about *cycle-accurate control*, which is exactly the mindset needed for pipeline-stage timing diagrams, hazard windows, and stall/flush logic in a bigger core — you've had to reason about "what bit position is this signal valid at" instead of "what pipeline stage", which is the same discipline in miniature.
- **A/B baseline preservation** (`_unmod` files, `USE_STOC_ALU` parameter, keeping stock `serv_alu` untouched) is exactly the regression-safety instinct DV teams want: never let a new feature risk the previously-verified path.

- For anything they probe on branch prediction/OoO/caches/MMU specifically, be honest that it's outside this project's scope, and pivot to general verification methodology you can speak to independently: constrained-random stimulus, coverage-driven verification, assertion-based verification (SVA), UVM scoreboards/reference models, RVFI/lockstep ISA-level checking (which SERV itself supports via `serv_debug.v` — you can speak to *what it's for* even though you didn't author it), and formal property checking.

PART 7 — RISC-V ISA Quick-Refresh (likely to come up independently)

- RV32I instruction formats: R/I/S/B/U/J, and which fields each carries (`opcode`, `funct3`/`funct7`, `rd`/`rs1`/`rs2`, immediate encodings — note B/J-type immediates are *not* contiguous, they're bit-shuffled for hardware convenience, which your `serv_immdec` has to unpack).
- Machine-mode CSR basics: `mstatus`, `mie`, `mcause`, `mepc`, `mtvec`, trap entry/exit (`mret`), timer interrupt (`mtip`) — all present in `serv_csr.v`.
- Load/store alignment and byte-lane handling — SERV's `serv_mem_if`/`serv_aligner` do this explicitly with a `mem_misalign` trap path, useful to know if asked about memory ordering/exception precision.
- RVFI (RISC-V Formal Interface): standard signal set (`rvfi_valid`, `rvfi_insn`, `rvfi_pc_rdata`/`wdata`, `rvfi_mem_*`, `rvfi_rd_*`) used to lockstep-compare an RTL core against a golden ISA simulator (e.g. Spike) instruction-by-instruction — SERV supports this natively (`RVFI_ENABLED`), and this is precisely the kind of methodology you should expect to use heavily at Qualcomm on real superscalar cores.

File saved for your own reference/reuse before the interview.